COMP1009
Programming Paradigms

Lecture OO-11

Parametric Polymorphism
and Boxing

This Lecture

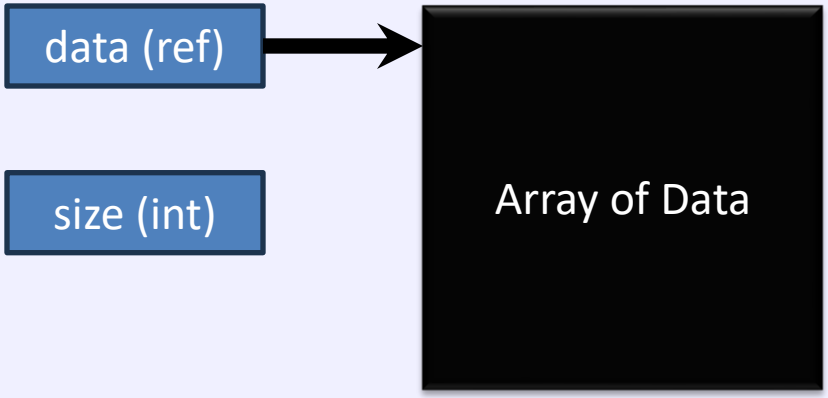
- Growable Array Class
- Parametric Polymorphism (generics)
  - and the limitations in Java
- Type Erasure
- Boxing and autoboxing
- Using classes other than Object

An Array Class (part 1)

```
public class ArrayInt
{
   private int size = 0;
   private int[] data = new int[0];

   public void add( int val )
   {
      int[] newData = new int[size+1];
      if ( size > 0 ) // then copy old contents
         System.arraycopy( data, 0, newData, 0, size);
      newData[size] = val; // add new value
      size++;
      data = newData;
   }
}
```

Array of ints



- We start with an array of data
- An object reference, called data, referencing it
- And a size variable with entry count

- We want to add an entry to the end…

add() creates a new array (copy) to use

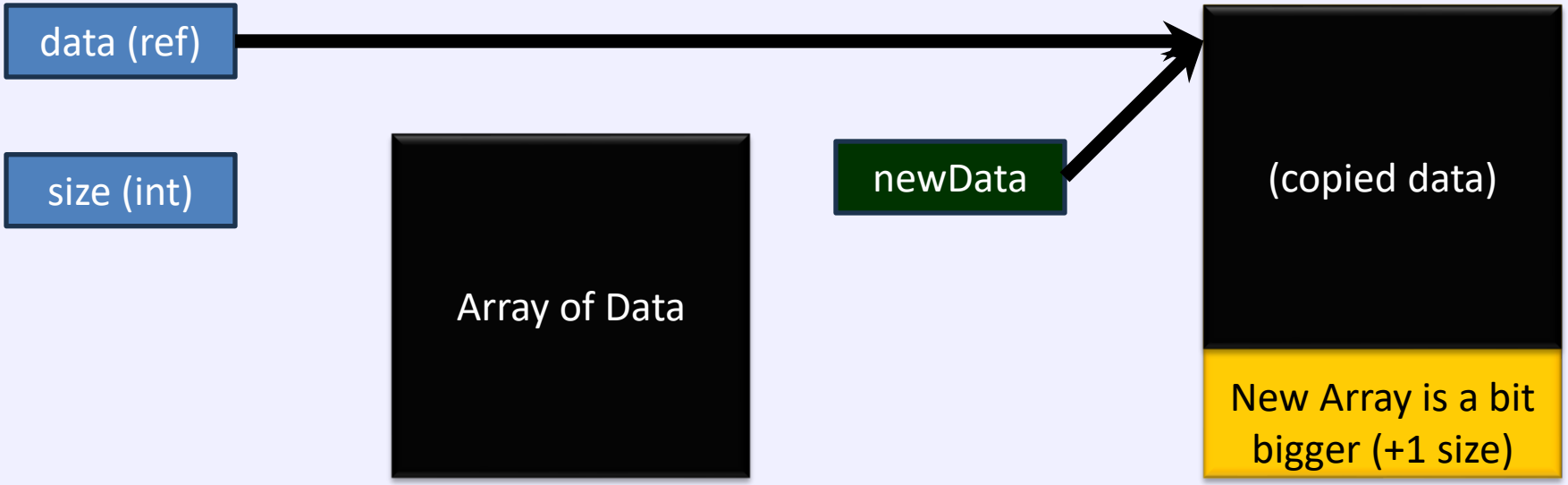
- We create a new array of data, with size+1 elements in it

  int[] newData = new int[size + 1];

- If previous array of data was not empty (**size > 0**), copy the contents over

  System.*arraycopy*(data, 0, newData, 0, size);

Copies in new entry and changes members



- Copy in the new value : **newData[size] = val**
- Increase size variable: **size++**
- Data variable needs to refer to the new array: **data = newData**
- Let old array of data get garbage collected

An Array Class (part 2)

```
...

   // Print everything in array
   public void printAll()
   {
      for ( int i = 0 ; i < data.length ; i++ )
         System.out.print( " "+ data[i] );
      System.out.println();
   }

   public static void main(String[] args)
   {
      ArrayInt arr = new ArrayInt();
      arr.add( 1 );
      arr.add( 2 );
      arr.printAll();
   }
}
```

Enhancing these

- This array can only contain integers
- What if I want to create arrays of doubles
- Or arrays of strings?

- We could copy-paste the code and change it to use the other types…

Part of original array with int

```
public class ArrayInt
{
   private int size = 0;
   private int[] data = new int[0];

   public void add( int val )
   {
      int[] newData = new int[size+1];
      if ( size > 0 )
         System.arraycopy( data, 0, newData, 0, size);
      newData[size] = val;
      size++;
      data = newData;
   }
}
```

int is stored
Array of ints

Part of an Array for doubles

```
public class ArrayDouble
{
   private int size = 0;
   private double[] data = new double[0];

   public void add( double val )
   {
      double [] newData = new double[size+1];
      if ( size > 0 )
         System.arraycopy( data, 0, newData, 0, size);
      newData[size] = val;
      size++;
      data = newData;
   }
}
```

Note: size is still an int

double is stored
Array of doubles

Part of an Array for Strings

```
public class ArrayString
{
   private int size = 0;
   private String[] data = new String[0];

   public void add(String val )
   {
      String[] newData = new String[size+1];
      if ( size > 0 )
         System.arraycopy( data, 0, newData, 0, size);
      newData[size] = val;
      size++;
      data = newData;
   }
}
```

String is stored
Array of strings

A generic array

Parametric Polymorphism

Key point of the lecture…

- Most OO languages support parametric polymorphism
- We could use parametric polymorphism to make a ***generic*** class which can store ANY type of object
- **One class** which is ***parameterised*** on **another class/type** (or multiple types)
- In Java there are some limitations, which are harder to understand than the key principle of 'Generics'
  1. Can only parameterise on a ***sub-class of a specific type***
     - All classes are subclasses of Object so we can use Object
  2. It is implemented using type erasure
     - *What is this? What effects does it have? Why do I care?*

A better way for a Generic class

- **Parameterise** our original class to avoid having to create different types **and also** check the types at **compile** time
  - We can use 'generics' – parameterised types
  - We need to say which **type** it stores when we create each object
  - E.g. **`GenArray<String> arr2`** : stores **`Strings`**

- Add placeholder <T> (or other label) after the class name

```
public class GenArray<T>
```

- Change the generic type from **int** to **T** throughout – for the contents!

```
private int size = 0; // NOT this int!!!
private T[] data = new T[0];
public void add(T val ) { ... }
```

Part of a Generic Array for type T

```
public class GenArray<T>
{
   private int size = 0;
   private T[] data = new T[0];

   public void add(T val )
   {
      T [] newData = new T[size+1];
      if ( size > 0 )
         System.arraycopy( data, 0, newData, 0, size);
      newData[size] = val;
      size++;
      data = newData;
   }
}
```

Note: there is a problem with this, as we will see, but this is what we want to do in theory/concept

```
GenArray<String> arr2 =
      new GenArray<String>();
```

Create one of these where T becomes String – like a replace of param T by String

Creating an array for Strings

```
GenArray<String> arr2
        = new GenArray<String>();
arr2.printAll();
arr2.add("1");
arr2.printAll();
arr2.add("2");
arr2.add("4");
arr2.printAll();
```

- arr2 knows that it stores String objects
- So, add expects a String and will tell you if you try to give it anything else

Problem 1: Type Erasure

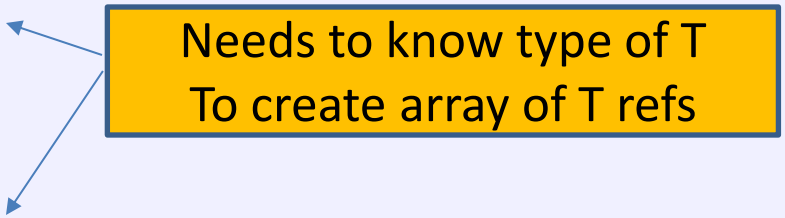
The problem of type erasure

- Generic types are implemented in Java using **'Type Erasure'** (type is 'erased' during compilation)
  - Type **exists at compile time**, **but not at runtime**
  - Type is 'erased' during compilation, so runtime sees it as some base-class type (e.g. Object)
    - Class 'Object' by default, but you can choose another
- This causes us a problem : removing the type at compilation means that **you cannot write code which needs to know the real type at runtime**
  - You cannot create an object of type T
  - You cannot create an array of object refs of type T
  - The compiler does not know what T will be at runtime!

Creating arrays of T object references

```
public class GenArray<T>
{
   private int size = 0;
   private T[] data = new T[0];

   public void add( T val )
   {
      T [] newData = new T[size+1];
      if ( size > 0 )
         System.arraycopy( data, 0, newData, 0, size);
      newData[size] = val;
      size++;
      data = newData;
   }
}
```



A fix – use the base class…

```
public class GenArray<T> // T extends Object
{
   private int size = 0;
   private Object[] data
             = new Object[0];

   public void add(T val )
   {
      Object [] newData = new Object[size+1];
      if ( size > 0 )
         System.arraycopy( data, 0, newData, 0, size);
      newData[size] = val;
      size++;
      data = newData;
   }
}
```

Whatever type T is, it **IS-An** Object, so create array of Objects instead

Still T here!

The other part – no change to printAll

```
public class GenArray<T>
{
   private int size = 0;
   private Object[] data
             = new Object[0];

   // Print everything in array
   public void printAll()
   {
     for ( int i = 0 ; i < data.length ; i++ )
       System.out.print( " "+ data[i] );
     System.out.println();
   }
}
```

Now it works

Type T is not used at runtime, so this will work regardless of type

Next problem

Extracting the data

Another problem

```
public class GenArray<T>
{
  private int size = 0;
  private Object[] data = new Object[0];

  public T get( int i )
  {
    return data[i];
  }

  public void add(T val )
  {
    …
```

Problem: we want a T and we have an array of objects

This problem was caused by using Object instead of T

Another solution

```
public class GenArray<T>
{
  private int size = 0;
  private Object[] data = new Object[0];

  public T get( int i )
  {
    return (T) data[i];
  }

  public void add(T val )
  {
    …
```

In this case we can safely cast the object – because we know they are all of type T, from when we stored them

A simpler generic class

It’s not always a problem…

Sometimes it is straightforward

```
public class Wrapper<T>
{
     T data;

     Wrapper( T init ) { data = init; }

     T get() { return data; }
     void set( T val ) { data = val; }

     public static void main(String[] args)
     {
          Wrapper<String> s = new Wrapper<>("Hello");
          Wrapper<Integer> i = new Wrapper<>(4);
          System.out.println( s.get() + i.get() );
     }
}
```

Final problem

Basic Data Types

(these are NOT Objects)

What about int, float etc?

```
GenArray<int> arr2
          = new GenArray<int>();
arr2.printAll();
arr2.add(1);
arr2.printAll();
arr2.add(2);
arr2.add(4);
arr2.printAll();
```

This will not work!
int is not an Object

- **int is NOT a class or an Object!**

Wrapping Classes

- int, boolean, float etc are NOT classes
- For each basic data type there is a wrapper class, which will wrap up the data in an object
  - Integer wraps up an int
  - Float wraps up a float
  - Etc, note the capital letter on the class type
- Boxing: wrapping up a data type in an object
- Unboxing: extracting the data again
- Auto-boxing: automatically doing this

A version which will store ints…

```
GenArray<Integer> arr3
          = new GenArray<Integer>();
arr3.printAll();
arr3.add(1); // Note: added an int!
arr3.printAll();
arr3.add(2);
arr3.add(4);
arr3.printAll();
```

- **Integer IS a class, and an 'Object' subclass!**
  - **int is not a class, not an Object subclass**

Autoboxing

- Whenever an object is needed, and you have a basic data type, **the compiler will create the code to convert to the wrapper class automatically** (if it can work out what to do)
- Because of this, **in many cases you can treat basic data types as if they are objects**
  - But they are not!
- When you need to specify an object type, e.g. with generics, you need to use the wrapper class, not the basic data type, e.g. **`Integer`**

Final comments

Final comments…

- We can also use a different base class than Object

```
public class GenArray2<T extends BaseClass>
{
   private int size = 0;
   private BaseClass[] data = new BaseClass[0];

   public void add( T val )
   {
      BaseClass[] newData = new BaseClass[size+1];
      if ( size > 0 ) …
```

- There are many system classes which are generics

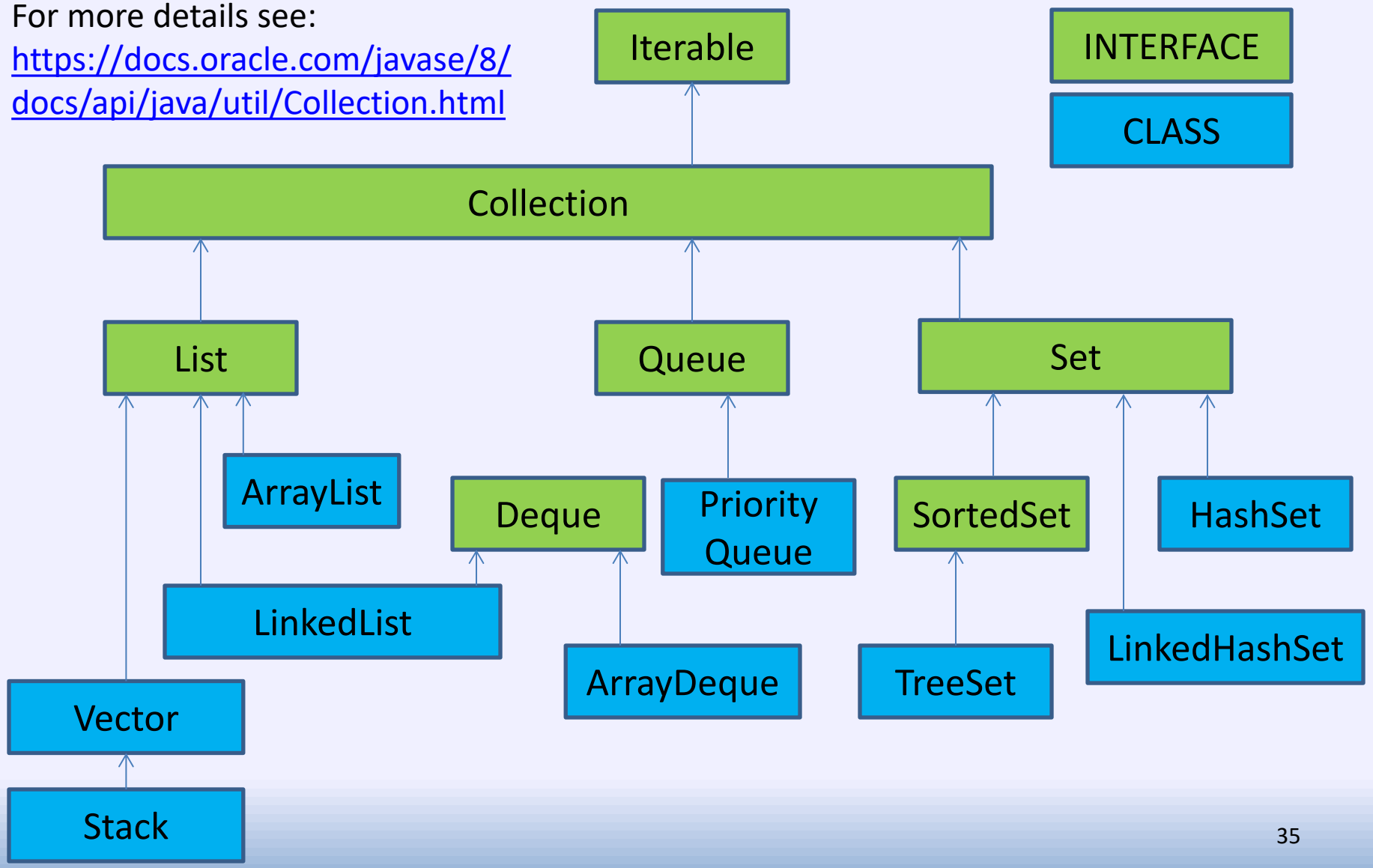
```
ArrayList<Integer> arr3 = new ArrayList<Integer>();
arr3.add(1);
arr3.add(2);
for ( int i = 0 ; i < arr3.size() ; i++ )
   System.out.print( " " + arr3.get(i) );
```

Summary

- You can create classes which are generic
- This is Parametric Polymorphism
  - When Haskell programmers talk about Polymorphism, they mean this sort of polymorphism
- You can create instances of a class which work with a specific other class (or multiple classes!)
- The generic class has to do exactly the same thing regardless of the parameter class type
- The real class will not be known at runtime
- You CAN use the class type at compile time
  - E.g. compiler can check you only add the right things to an array
- Java's collection classes are Generics…

Java's collection classes are generics

For more details see:
https://docs.oracle.com/javase/8/docs/api/java/util/Collection.html



Next Lecture – lecture 12

- Some comments about Java's collection classes
  - See previous slide
- Generic Lists (and inner classes)
- Iterator pattern